Practical 1

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace std::chrono;

// Bubble Sort
void bubbleSort(vector<int>& arr)
{
    int n = arr.size();

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
}

// Selection Sort
void selectionSort(vector<int>& arr)
{
    int n = arr.size();
```

```
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;

        swap(arr[i], arr[minIndex]);
    }
}

// Insertion Sort
void insertionSort(vector<int>& arr)
{
    int n = arr.size();

    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
```

```
        }

        arr[j + 1] = key;
    }
}

// Merge Function
void merge(vector<int>& arr, int left, int mid, int right)
{
    vector<int> temp;

    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right)
    {
        if (arr[i] <= arr[j])
            temp.push_back(arr[i++]);
        else
            temp.push_back(arr[j++]);
    }

    while (i <= mid)
        temp.push_back(arr[i++]);

    while (j <= right)
```

```
        temp.push_back(arr[j++]);

    for (int k = 0; k < temp.size(); k++)
        arr[left + k] = temp[k];
}

// Merge Sort
void mergeSort(vector<int>& arr, int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

// Partition for Quick Sort
int partition(vector<int>& arr, int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}

// Quick Sort
void quickSort(vector<int>& arr, int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition(arr, low, high);

        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

```
int main()
{
    const int n = 100;

    vector<int> arr(n), temp;

    srand(time(0));

    // Generate random numbers
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 1000;

    cout << "Number of Elements = " << n << "\n\n";

    // Bubble Sort
    temp = arr;
    auto start = high_resolution_clock::now();
    bubbleSort(temp);
    auto stop = high_resolution_clock::now();

    cout << "Bubble Sort Time    : "
        << duration_cast<microseconds>(stop - start).count()
        << " microseconds\n";

    // Selection Sort
    temp = arr;
```

```
start = high_resolution_clock::now();
selectionSort(temp);
stop = high_resolution_clock::now();

cout << "Selection Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds\n";

// Insertion Sort
temp = arr;
start = high_resolution_clock::now();
insertionSort(temp);
stop = high_resolution_clock::now();

cout << "Insertion Sort Time : "
    << duration_cast<microseconds>(stop - start).count()
    << " microseconds\n";

// Merge Sort
temp = arr;
start = high_resolution_clock::now();
mergeSort(temp, 0, n - 1);
stop = high_resolution_clock::now();

cout << "Merge Sort Time     : "
    << duration_cast<microseconds>(stop - start).count()
```

```
        << " microseconds\n";

    // Quick Sort
    temp = arr;
    start = high_resolution_clock::now();
    quickSort(temp, 0, n - 1);
    stop = high_resolution_clock::now();

    cout << "Quick Sort Time     : "
        << duration_cast<microseconds>(stop - start).count()
        << " microseconds\n";

    return 0;
}
```

Output:

```
Number of Elements = 100

Bubble Sort Time    : 82 microseconds
Selection Sort Time : 42 microseconds
Insertion Sort Time : 29 microseconds
Merge Sort Time     : 89 microseconds
Quick Sort Time     : 13 microseconds
```